# Time Matters Appointment Scheduler Technical Report

Software Design 1 SDN150S | Project 12

Prepared for Management | 7 November 2026

Group 12

Noluthando Basi | 260874574

Zilungile Cwathi | 250513447

Phakamani Nicholas Dlamini | 261059742

Sekete Manthata | 260555770

Liyabona Ndevu | 260173436

Okuhle Ntweni | 260016039

Reneloe Wondeful Phakwago | 260633283

Lisakhanya Songqwaba | 261136437

Repository: https://github.com/ZuluBarbie/AppointmentScheduler

## 1 Introduction

Time Matters is a local Java appointment scheduler for Group 12. It supports appointment creation, editing, deletion, reminders and daily, weekly and monthly views, as required for Project 12 (Cape Peninsula University of Technology, 2026). Recurrence, clash detection and calendar export extend this workflow. Management can use this report to assess the design, verified behaviour and operating limitations.

## 2 Sub questions

How can one saved appointment series support three schedule views? How can invalid input and overlapping appointments be rejected without losing data? How can reminders alert once per occurrence during a session? How can the displayed schedule be exported for calendar import?

## 3 Discussion

The design separates the interface from appointment rules, storage, reminders and export. Constructors create valid objects; private fields protect internal state; public methods expose operations. Conditional statements, loops and switch expressions implement validation, recurrence and date filtering. SchedulerFrame retains its paired NetBeans JFrame Form. NewJFrame1 is the welcome JFrame, built in Java with GridBagLayout; it has no visual form file.

Development used incremental milestones: a basic window, appointment entry, editing and storage, recurrence and reminders, then calendar export. The final interface adds the Time Matters welcome screen, cyan accents and a light scheduler workspace. Its Submission files button opens the supplied documents folder. NetBeans 31 supports editing and running; JDK 21 release compilation checks language and standard-library compatibility. GitHub records meaningful changes throughout development.

Appointment validates titles, dates, times, repeat ranges and reminder lead times. AppointmentBook stores each series once and expands dated Occurrence records for the selected view. Weeks run Monday to Sunday. Selecting an occurrence allows the underlying series to be edited or deleted. Daily and weekly recurrence ends inclusively, within 366 days of its start.

Before saving, the book rejects overlapping time intervals on a common occurrence date. Adjacent appointments are allowed. It writes a temporary file before replacing the saved file and rejects a save when another window has changed that file (Oracle, n.d.). Reminders are checked every second and tracked once per occurrence per open session.

CalendarExport writes the displayed occurrences as iCalendar events. Local times are converted to UTC. Text escaping and line folding follow RFC 5545 (Desruisseaux, 2009). Export produces a file for import, rather than automatic calendar synchronisation.

## 4 Pseudocode

1. Start the application and show the Time Matters welcome window.

2. When Open scheduler is selected, open the scheduler and close the welcome window.

3. When Submission files is selected, open the folder containing the supplied documents.

4. Load saved appointments. If loading fails, preserve the file, show the error and disable saving.

5. Display appointments for the selected day, week or month, ordered by date and start time.

6. When the view or navigation date changes, calculate its date range and refresh the displayed occurrences.

7. To add an appointment, read its title, date, times, location, repeat settings and reminder minutes.

8. Reject missing or invalid values, an end time not after the start, or an invalid repeat range.

9. Compare candidate occurrences with other appointments. Reject overlapping times on a shared date.

10. If valid, save the changed list safely and refresh the view. Show a message if saving fails.

11. When a row is selected, copy its series details into the fields and enable editing and deletion.

12. To edit, validate the replacement and check clashes while excluding the original series. Save and refresh.

13. To delete, ask for confirmation. Remove the selected appointment or entire series, save and refresh.

14. To clear the selection, reset the entry fields without deleting stored appointments.

15. While the scheduler is open, check upcoming occurrences every second. Show each due reminder once per session.

16. To export, take a snapshot of the displayed occurrences and ask for a destination file.

17. Confirm replacement of an existing file, then write one calendar event per occurrence. Report errors visibly.

18. When the scheduler closes, stop its reminder timer.

## 5 UML class diagram

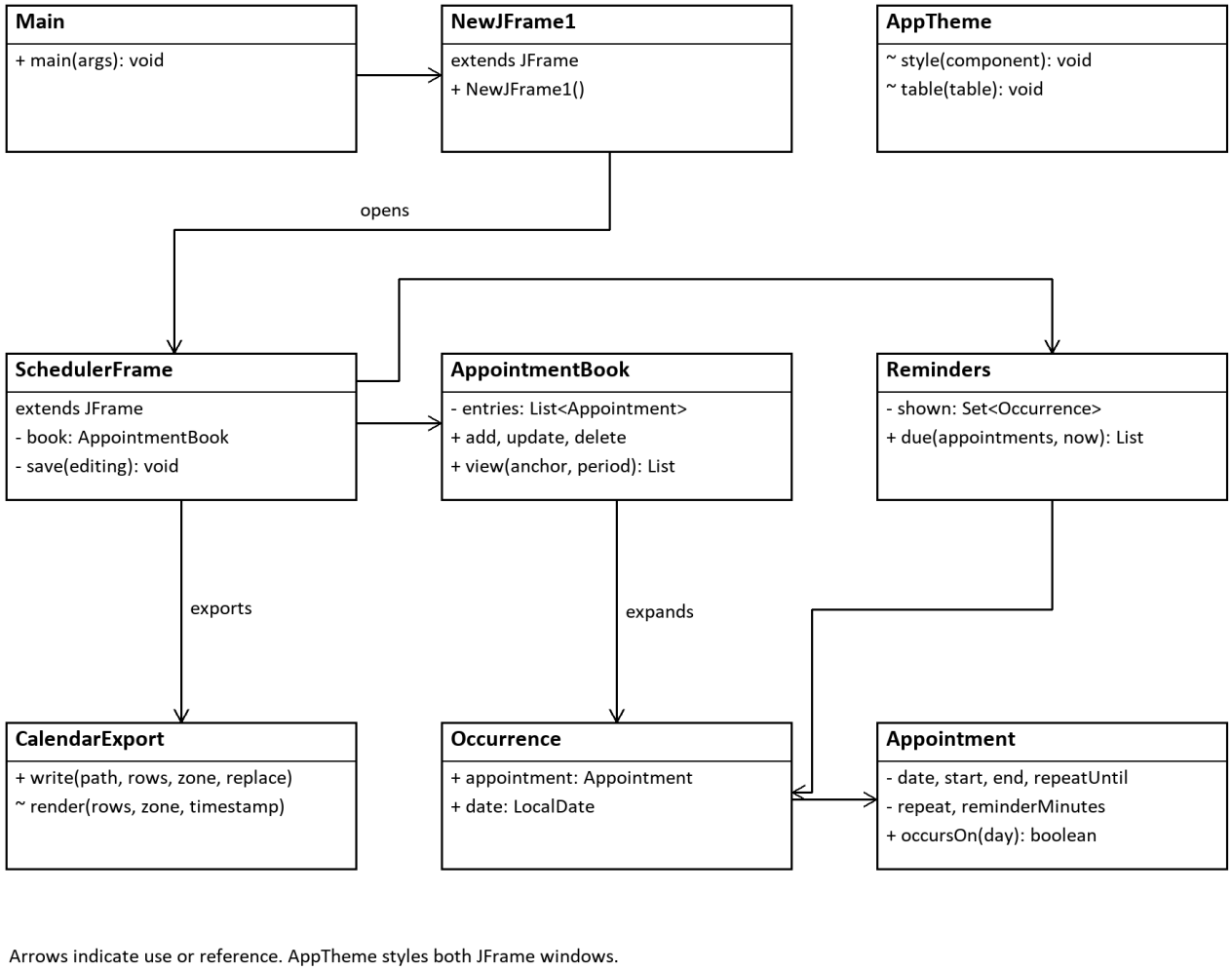



**Figure 1. Main opens the welcome JFrame. The scheduler uses separate storage, reminder and export classes. Occurrence references its saved Appointment.**

## 6 Results and analysis

On Windows, the NetBeans Ant build and standalone build succeeded. All 64 automated checks passed on 7 November. Checks cover input boundaries, file persistence, failed writes, date ranges, recurrence, overlap rejection, reminder timing and calendar export. Swing integration checks confirmed welcome navigation, saved appointment loading, all three views and selection of a row for editing. Verification left the normal appointment files unchanged.

Earlier export verification exercised the save chooser and inspected the resulting calendar file. No third-party calendar import was verified. The welcome and scheduler windows were rendered and inspected after styling changes. Local sample appointments support the demonstration; they are not evidence of group contributions.

## 7 Operating limitations

The app must remain open for reminders. Reopening during a reminder window can alert again. Editing or deleting a recurring row affects its whole series. Local storage is intended for one user and does not provide login or shared online access. Calendar export is a snapshot; repeated imports may create duplicates. A damaged data file must be corrected or restored before saving can resume.

## 8 Conclusion

Time Matters implements the required appointment operations and schedule views, with tested validation, persistence and reminders. The light workspace retains the welcome screen branding while keeping controls readable. The separate source project and complete Java listing accompany this report. The group must record the required demonstration and provide truthful evidence of participation for submission on 8 November 2026.

## 9 Java program

The complete nine-class Java program is supplied in TimeMatters_Complete_Java_Program.pdf and the editable NetBeans source project ZIP. The entry point is demo.Main. The excerpt below illustrates the recurrence rule from that program.

```
public boolean occursOn(LocalDate day) {
   if (day.isBefore(date) || day.isAfter(repeatUntil)) return false;
   return repeat != Repeat.WEEKLY
      || day.getDayOfWeek() == date.getDayOfWeek();
}
```

## References

Cape Peninsula University of Technology (2026) Software Design 1 SDN150S project brief. Course handout.

Desruisseaux, B. (ed.) (2009) Internet Calendaring and Scheduling Core Object Specification (iCalendar). RFC 5545. Available at: https://www.rfc-editor.org/rfc/rfc5545 (Accessed: 6 November 2026).

Oracle (n.d.) Files. Java SE 21. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html (Accessed: 4 November 2026).